Question 3(c) - Base Model Evaluation

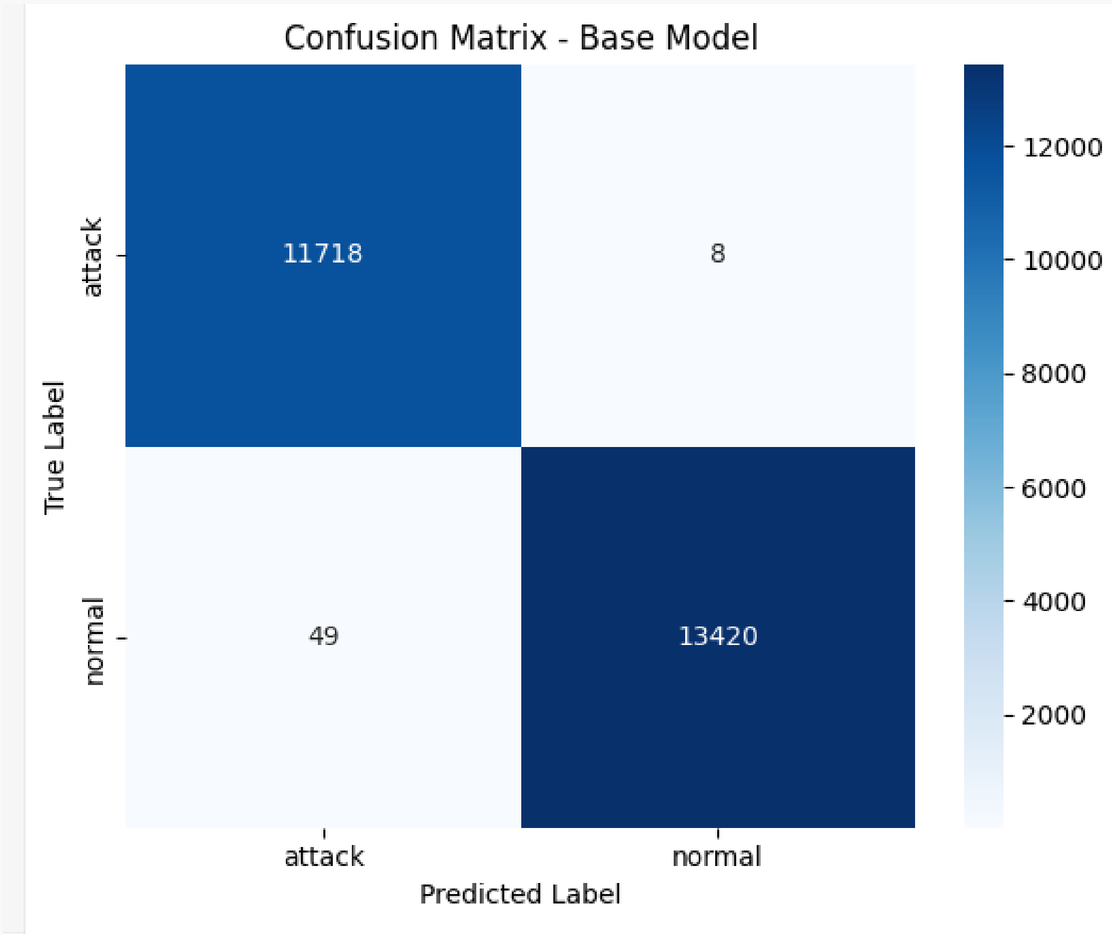
The base DNN model was evaluated on the test set X_test (25,195 samples). The model architecture consists of three hidden layers (128, 64, and 32 neurons with ReLU activation) and an output layer with 2 neurons and softmax activation, trained using the Adam optimizer with sparse categorical cross-entropy loss for 10 epochs.

The classification report shows that the base model achieves perfect precision, recall, and F1-score of 1.00 for both attack and normal classes, with an overall accuracy of 1.00 on the 25,195 test samples (11,726 attack and 13,469 normal). The confusion matrix shows 11,718 true attack predictions, 13,420 true normal predictions, 49 false positives, and 8 false negatives.

<u>Classification Report (base_model):</u>

```
788/788 [==============================] - 1s 626us/step
Classification Report (base_model):
              precision    recall  f1-score   support

      attack       1.00      1.00      1.00     11726
      normal       1.00      1.00      1.00     13469

    accuracy                           1.00     25195
   macro avg       1.00      1.00      1.00     25195
weighted avg       1.00      1.00      1.00     25195

Confusion Matrix (base_model):
[[11718     8]
 [   49 13420]]
```

Confusion Matrix (base_model):



Question 4(b) - Quantized TFLite Model Evaluation

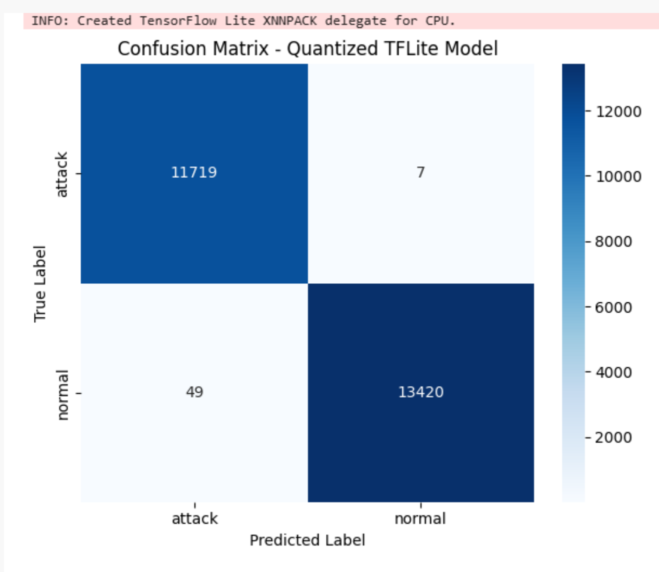
Dynamic Range Quantization was applied to the base model using the TensorFlow Lite converter with tf.lite.Optimize.DEFAULT. This converts model weights from float32 to int8, reducing model size while maintaining accuracy. The quantized model was evaluated on the same test set X_test using the TFLite interpreter.

The quantized model achieves identical classification report results to the base model: precision, recall, and F1-score of 1.00 for both classes and an overall accuracy of 1.00. The confusion matrix shows 11,719 true attack predictions, 13,420 true normal predictions, 49 false positives (normal predicted as attack), and 7 false negatives (attack predicted as normal). Compared to the base model, the quantized model has one fewer false negative (7 vs. 8), demonstrating that Dynamic Range Quantization preserves and even marginally improves classification performance while significantly reducing model size for deployment on the Arduino Nano 33 BLE Sense.

Classification Report (tflite_quant_model):

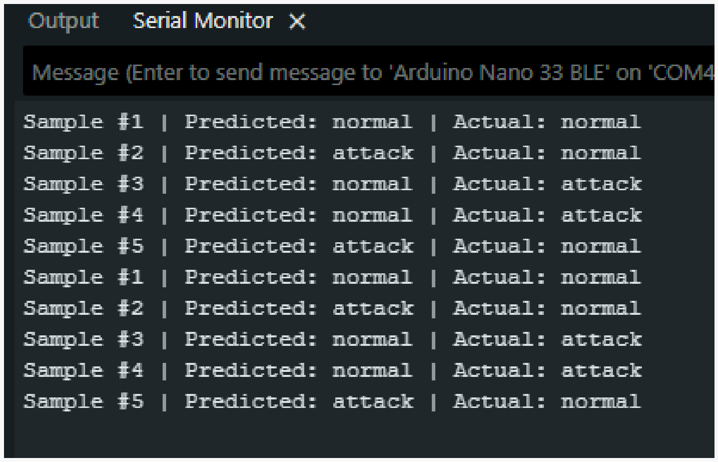
```
Classification Report (tflite_quant_model):
              precision    recall  f1-score   support

      attack       1.00      1.00      1.00     11726
      normal       1.00      1.00      1.00     13469

    accuracy                           1.00     25195
   macro avg       1.00      1.00      1.00     25195
weighted avg       1.00      1.00      1.00     25195

Confusion Matrix (tflite_quant_model):
[[11719     7]
 [   49 13420]]
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
```

Confusion Matrix (tflite_quant_model):



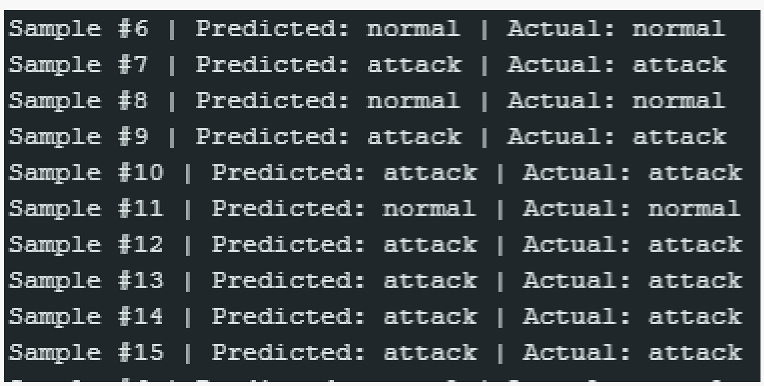
Question 5(c) - Serial Monitor Output: First 5 Samples

The network_model.h file generated from the quantized TFLite model was added to the network_data folder alongside network_data.ino. The sketch was uploaded to the Arduino Nano 33 BLE Sense. The Serial Monitor output for the first 5 test samples is shown below, displaying Sample #, Predicted Class, and Actual Class for each sample.



Question 5(d) - Serial Monitor Output: Next 10 Samples

Using the code at the end of the Python notebook, 10 additional feature vectors and actual labels were extracted from X_test (samples 6 through 15) and embedded in network_data.ino. The updated sketch was re-uploaded to the Arduino Nano 33 BLE Sense and the Serial Monitor output for these 10 samples is shown below.



The Serial Monitor outputs confirm that the deployed quantized model on the Arduino Nano 33 BLE Sense runs inference correctly on embedded test samples. The on-device predictions are consistent with the Python-based TFLite interpreter results, demonstrating successful deployment of a TinyML network anomaly detection model on resource-constrained hardware.